

Unit - II

- **Fundamentals of Fine-Tuning (Transfer Learning and Domain Adaptation)**
- **Full Fine-tuning vs. Feature-based Fine-tuning**
- **Dataset preparation and preprocessing for fine-tuning**
- **Evaluation metrics and validation techniques**

Introduction

- Fine-tuning, transfer learning, and
- Domain adaptation are key techniques in deep learning
- They allow models to reuse learned knowledge
- Reduce data requirements and training time
- Widely used in NLP and Computer Vision

What is Transfer Learning?

- Using a **pre-trained model** for a new but related task
- Avoids training models from scratch
- Effective when source and target tasks share similarities
- Common in deep learning models like CNNs and Transformers

Key Steps in Transfer Learning

1. Pre-training

- Model trained on a large dataset
- Learns general features
- Examples: ImageNet, large text corpora

2. Fine-tuning

- Model adapted to a specific task
- Trained on a smaller, task-specific dataset
- Uses lower learning rate

Transfer Learning Example

- **source Task:** Image classification on ImageNet
- **Target Task:** Medical image classification (X-rays)
- Pre-trained model already understands:
 - Edges
 - Shapes
 - Textures
- Fine-tuning helps specialize for medical data

Types of Transfer Learning

1. Feature Extraction

- Pre-trained layers are frozen
- Only new classifier is trained

2. Fine-Tuning

- Some pre-trained layers are unfrozen
- Model weights updated with small learning rate

What is Domain Adaptation?

- Specialized form of transfer learning
- Source and target domains differ in data distribution
- Task remains the same
- Useful when target domain has little or no labeled data

Domain Shift

- Difference in data distributions between domains

Example:

- Training data: sunny images
- Testing data: rainy images
- Can significantly affect model performance

Types of Domain Adaptation

- **Unsupervised Domain Adaptation**
 - Labeled data only in source domain
 - Target domain data is unlabeled
- **Supervised Domain Adaptation**
 - Labeled data available in both domains
 - Still requires reducing domain gap

Domain Adaptation Techniques

- Adversarial Training
- Domain-Adversarial Neural Networks (DANN)
- Self-Ensembling
- Data Augmentation

1. Adversarial Training

- A **domain discriminator** that tries to distinguish between source and target domains
- A **feature extractor** that learns features to confuse the discriminator

Result: **Domain-invariant feature representations**

❖ Adversarial training is a **learning technique** in which two models (or components of a model) are trained **against each other** so that one model improves by **challenging** the other.

Domain Adaptation Example

- **Source Domain:** Facial recognition trained on well-lit, frontal face images
- **Target Domain:** Faces captured in low-light conditions or at different angles
- Domain adaptation enables the model to perform reliably across both conditions.

Full Fine-Tuning vs Feature-Based Fine-Tuning

Full Fine-Tuning vs Feature-Based Fine-Tuning

1. Introduction

Fine-tuning is the process of adapting a pre-trained model to a specific task or domain. There are two common approaches:

Full Fine-Tuning

Feature-Based Fine-Tuning

Both methods aim to improve performance but differ in how much of the model is updated during training.

2. Full Fine-Tuning

Meaning

In full fine-tuning, all layers of the pre-trained model are updated using task-specific data.

How It Works

Start with a pre-trained model

Train the entire network on new domain/task data

All weights are adjusted via backpropagation

Advantages of Full Fine-Tuning

- Higher performance on complex tasks
- Learns task-specific and domain-specific features
- Better adaptation when target task differs

3. Feature-Based Fine-Tuning

Meaning

In feature-based fine-tuning, the pre-trained model is frozen, and only a new task-specific head is trained.

How It Works

Use the pre-trained model as a **feature extractor**

Extract embeddings from fixed layers

Train a lightweight classifier on top

Aspect	Full Fine-Tuning	Feature-Based Fine-Tuning
Layers Updated	All layers of the pre-trained model are updated	Only the task-specific output layer is trained
Compute Cost	High computational and memory cost	Low computational and memory cost
Data Requirement	Requires a large amount of labeled data	Works with a small labeled dataset
Domain Adaptation	Strong adaptation to the target domain	Limited adaptation to the target domain
Training Time	Long training time	Short training time
Overfitting Risk	Higher risk due to many trainable parameters	Lower risk as most parameters are frozen

Dataset preparation and preprocessing for fine-tuning

Dataset Preparation

Introduction

Fine-tuning is the process of adapting a pre-trained model to a specific task or domain by training it further on a custom dataset.

The quality of the dataset and preprocessing directly affects the model's accuracy and generalization.

- Data Collection
- Data Annotation / Labeling
- Dataset Splitting
- Data Balancing

Data Collection

Collect data relevant to the target task (text, images, audio, tabular data).

Sources:

- Public datasets (Kaggle, Hugging Face)
- Web scraping
- APIs
- Internal databases

Ensure:

- Domain relevance
- Sufficient data volume
- Legal and ethical compliance

Data Annotation / Labeling

Assign correct labels to data samples.

Types of labeling:

- **Classification** → class labels
- **Regression** → numeric values
- **Sequence labeling** → tags (NER, POS)

Tools:

- Label Studio
- CVAT
- Prodigy

Quality checks:

- Human validation
- Inter-annotator agreement

Dataset Splitting

Divide data into:

- **Training set** (70–80%)
- **Validation set** (10–15%)
- **Test set** (10–15%)

Purpose:

- Training → model learning
- Validation → hyperparameter tuning
- Test → final evaluation

Data Balancing

Avoid class imbalance which leads to biased models.

Techniques:

- Oversampling (SMOTE)
- Undersampling
- Class weighting
- Data augmentation

Data Preprocessing

- Data Cleaning
- Text Preprocessing
- Image Preprocessing
- Feature Engineering
- Tokenization & Encoding
- Data Formatting for Fine-Tuning
- Quality Checks

Data Cleaning

Remove:

- Duplicate records
- Incomplete or corrupted data

Handle missing values:

- Deletion
- Mean/median imputation
- Forward/backward fill

Text Preprocessing

- Lowercasing text
- Removing:
 - Punctuation
 - Special characters
 - URLs
- Tokenization
- Stop-word removal (optional)
- Stemming / Lemmatization
- Padding and truncation for fixed input length

Image Preprocessing

- Resize images to fixed dimensions
- Normalize pixel values
- Convert color channels (RGB/Grayscale)
- Data augmentation:
 - Rotation
 - Flipping
 - Cropping
 - Brightness adjustment

Feature Engineering

- Convert raw data into meaningful features.
- Examples:
 - One-hot encoding
 - TF-IDF
 - Word embeddings
 - Normalization / Standardization

Tokenization & Encoding

- Convert data into numerical form.
- Use:
 - Word-level tokenizers
 - Sub-word tokenizers (BPE, WordPiece)
 - Map tokens to IDs using tokenizer vocabulary.

Data Formatting for Fine-Tuning

Align dataset format with model requirements:

- Input-output pairs
- JSON / CSV / TFRecord

Example (NLP):

```
{  
  "input": "What is IoT?",  
  "output": "Internet of Things connects devices via internet."  
}
```

Quality Checks

- Remove noisy samples
- Validate labels
- Detect outliers
- Check dataset consistency

Evaluation Metrics and Validation Techniques

Evaluation Metrics and Validation Techniques

Introduction

After a machine learning or deep learning model is trained, we must evaluate how well it performs. Training accuracy alone is not enough because a model may perform well on training data but fail on new, unseen data. This problem is known as over fitting.

To overcome this:

Evaluation metrics are used to measure model performance numerically.

Validation techniques are used to check how well the model generalizes to unseen data.

Together, they help in building reliable, accurate, and real-world usable models.

Evaluation Metrics

Evaluation metrics are **quantitative measures** that assess the quality of predictions made by a model.

Different machine learning tasks require **different metrics**:

- Classification → Accuracy, Precision, Recall, F1-score
- Regression → MAE, MSE, RMSE
- NLP → BLEU, Perplexity

Choosing the wrong metric can lead to **incorrect conclusions** about model performance.

Evaluation Metrics for Classification Problems

Classification problems involve predicting **discrete class labels** (e.g., Yes/No, Spam/Not Spam).

- Confusion Matrix
- Accuracy
- Precision
- Recall (Sensitivity)
- F1-Score
- ROC Curve and AUC

Confusion Matrix

A confusion matrix is a **tabular representation** that shows how predictions compare with actual values.

Actual \ Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Explanation of Terms

- **True Positive (TP):** Model correctly predicts a positive class
- **True Negative (TN):** Model correctly predicts a negative class
- **False Positive (FP):** Model predicts positive but actual is negative
- **False Negative (FN):** Model predicts negative but actual is positive

The confusion matrix is important because:

- It gives a **complete picture** of model performance
- All other classification metrics are derived from it

Accuracy

- Accuracy measures the **overall correctness** of the model.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Interpretation

Accuracy tells us the percentage of total predictions that are correct.

Advantages

Simple and intuitive

Useful when classes are **balanced**

Limitations

Misleading when data is **imbalanced**

High accuracy does not always mean good performance

Example: In a dataset with 95% negative samples, predicting everything as negative gives 95% accuracy but poor usefulness.

Precision

Precision measures **how reliable positive predictions are**.

$$Precision = \frac{TP}{TP + FP}$$

Interpretation

Out of all predicted positives, how many are actually correct?

Use Case

Important when **false positives are costly**

Example:

- Spam detection

- Fraud detection

High precision means fewer false alarms.

Recall (Sensitivity)

Recall measures **how many actual positives are correctly identified**.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation

Out of all actual positives, how many did the model detect?

Use Case

Important when **false negatives are dangerous**

Example:

- Cancer detection

- Security threat detection

High recall means fewer missed cases.

F1-Score

F1-score is the **harmonic mean** of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Why Harmonic Mean?

Penalizes extreme values

Balances precision and recall

Advantages

Useful for **imbalanced datasets**

Gives a single performance measure

Specificity

Specificity measures how well the model identifies **negative samples**.

$$\textit{Specificity} = \frac{TN}{TN + FP}$$

Interpretation

Out of all actual negatives, how many were correctly predicted?

Use Case

Medical diagnosis

Fault detection systems

ROC Curve and AUC

ROC Curve

Graph between:

True Positive Rate (Recall)

False Positive Rate

AUC (Area Under Curve)

Measures model's ability to distinguish between classes

Value ranges from **0 to 1**

Interpretation

$AUC = 1 \rightarrow$ Perfect model

$AUC = 0.5 \rightarrow$ Random guessing

Higher AUC means better classification performance.

Thank You